



BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS



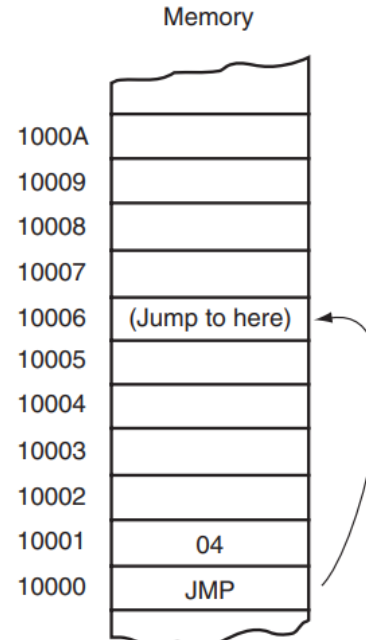
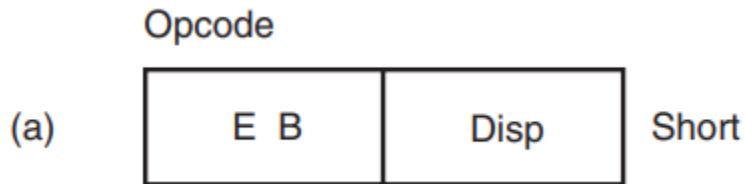
Program Control Instructions

- Direct the flow of a program and allow the flow to change.
- A change in flow often occurs after a decision made with the **CMP or TEST** instruction is followed by a conditional jump instruction

Unconditional Jump (JMP)

- Short jumps are called relative jumps because they can be moved, along with their related software, to any location in the current code segment without a change.

- Displacement between +127 and -128
- $IP = IP + \text{Sign Extend}(\text{Displacement})$

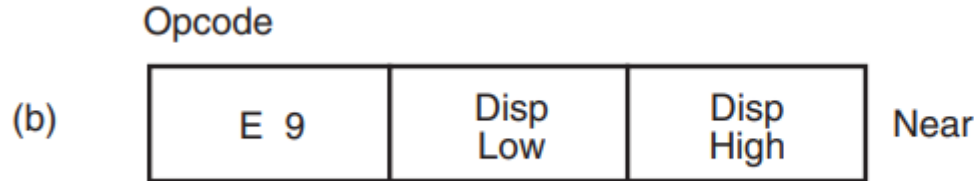


A short jump to four memory locations beyond the address of the next instruction

CS = 1000H
IP = 0002H
New IP = IP + 4
New IP = 0006H

Near JUMP

- The near jump is similar to the short jump, except that the distance is farther.
- A near jump passes control to an instruction in the current code segment located **within $\pm 32K$ bytes** from the near jump instruction
- The near jump is a 3-byte instruction that contains an opcode followed by a signed 16-bit displacement.



Near Jump

- A near jump that adds the displacement (0002H) to the contents of IP.

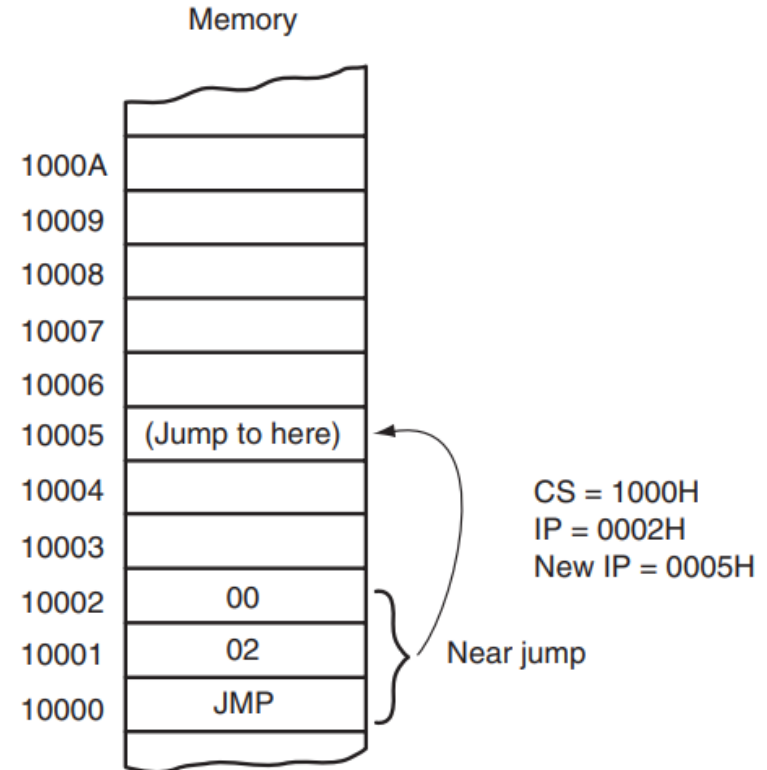
```

        XOR    BX, BX
START:  MOV    AX, 1
        ADD    AX, BX
        JMP    NEXT
    
```

<skipped memory locations>

```

NEXT:   MOV    BX, AX
        JMP    START
    
```



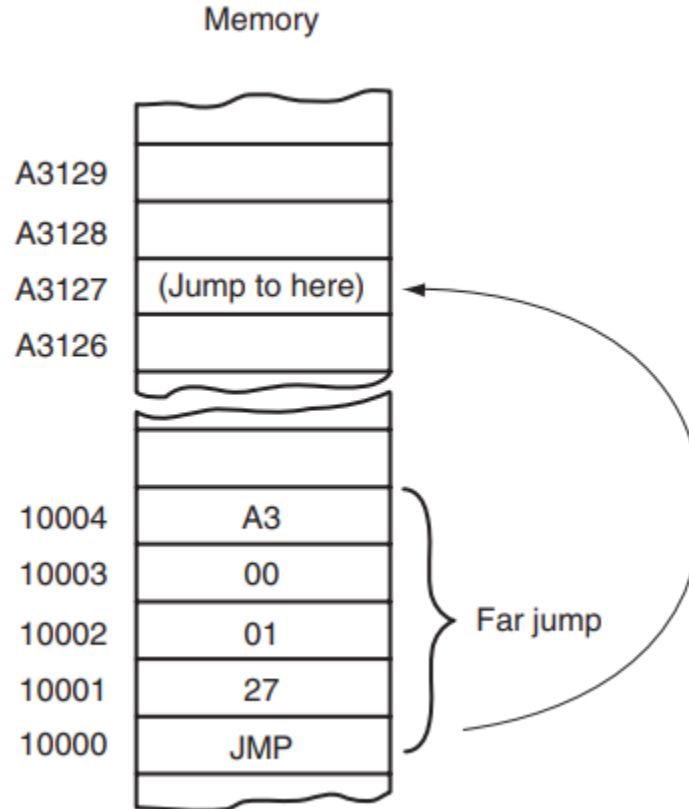
Far Jump

- A far jump instruction obtains a new segment and offset address to accomplish the jump.
- Bytes 2 and 3 of this 5-byte instruction contain the new offset address; bytes

Opcode



Far Jump



JUMP Instruction

Opcode



Opcode



Opcode



INTRASEGMENT JUMPS

INTERSEGMENT JUMPS

Jumps with Register Operands

- The jump instruction can also use a 16- or 32-bit register as an operand

E.g. `JMP AX`

Copies the contents of the `AX` register into the `IP` when the jump occurs.

Indirect Jumps Using an Index

- The jump instruction may also use the [] form of addressing to directly access the jump table.

E.g. `JMP TABLE [SI]`

Conditional Jump

- Conditional jump instructions **are always short jumps** in the 8086
- This limits the range of the jump to within **+127 bytes and -128 bytes** from the location following the conditional jump
- The conditional jump instructions test the following flag bits: sign (S), zero (Z), carry (C), parity (P), and overflow (O).
- If the condition under test is true, a branch to the label associated with the jump instruction occurs.
- If the condition is false, the next sequential step in the program executes.

E.g. JC Jump if carry is set
 JNE Jump if not equal



BITS Pilani
K K Birla Goa Campus

Assembly Language	Tested Condition	Operation
JA	$Z = 0$ and $C = 0$	Jump if above
JAE	$C = 0$	Jump if above or equal
JB	$C = 1$	Jump if below
JBE	$Z = 1$ or $C = 1$	Jump if below or equal
JC	$C = 1$	Jump if carry
JE or JZ	$Z = 1$	Jump if equal or jump if zero
JG	$Z = 0$ and $S = 0$	Jump if greater than
JGE	$S = 0$	Jump if greater than or equal
JL	$S \neq 0$	Jump if less than
JLE	$Z = 1$ or $S \neq 0$	Jump if less than or equal
JNC	$C = 0$	Jump if no carry
JNE or JNZ	$Z = 0$	Jump if not equal or jump if not zero
JNO	$O = 0$	Jump if no overflow
JNS	$S = 0$	Jump if no sign (positive)
JNP or JPO	$P = 0$	Jump if no parity or jump if parity odd
JO	$O = 1$	Jump if overflow
JP or JPE	$P = 1$	Jump if parity or jump if parity even
JS	$S = 1$	Jump if sign (negative)
JCXZ	$CX = 0$	Jump if CX is zero
JECXZ	$ECX = 0$	Jump if ECX equals zero
JRCXZ	$RCX = 0$	Jump if RCX equals zero (64-bit mode)

LOOP

- The LOOP instruction is a combination of a **decrement CX** and the **JNZ conditional jump**.
- In the 8086 through the 80286 processors, LOOP decrements CX;
 - if $CX \neq 0$, it jumps to the address indicated by the label.
 - if CX becomes 0, the next sequential instruction executes.
- In the 80386 and above, LOOP decrements either CX or ECX, depending upon the instruction mode.

Example

```

;A program that sums the contents of BLOCK1 and BLOCK2
;and stores the results on top of the data in BLOCK2.
;
.MODEL SMALL                ;select SMALL model
.DATA                      ;start data segment
BLOCK1 DW 100 DUP(?)        ;100 words for BLOCK1

BLOCK2 DW 100 DUP(?)        ;100 words for BLOCK2

.CODE                      ;start code segment
.STARTUP                  ;start program
    MOV AX,DS              ;overlap DS and ES
    MOV ES,AX
    CLD                    ;select auto-increment
    MOV CX,100             ;load counter
    MOV SI,OFFSET BLOCK1   ;address BLOCK1
    MOV DI,OFFSET BLOCK2   ;address BLOCK2
L1:    LODSW                ;load AX with BLOCK1
    ADD AX,ES:[DI]          ;add BLOCK2
    STOSW                   ;save answer
    LOOP L1                 ;repeat 100 times

.EXIT
END

```

Conditional LOOPS

- **LOOPE (loop while equal)** instruction jumps if $CX \neq 0$ while an equal condition exists.
 - It will exit the loop if the condition is not equal or if the CX register decrements to 0.
- **LOOPNE (loop while not equal)** instruction jumps if $CX \neq 0$ while a not-equal condition exists.
 - It will exit the loop if the condition is equal or if the CX register decrements to 0.

Procedures

- A procedure is a **reusable section** of the software that is stored in memory once, but used as often as necessary.
- The CALL instruction links to the procedure, and the RET (return) instruction returns from the procedure.
- The stack stores the return address whenever a procedure is called during the execution of a program.
- The CALL instruction pushes the address of the instruction following the CALL (return address) on the stack.
- The RET instruction removes an address from the stack so the program returns to the instruction following the CALL
- A procedure begins with the PROC directive and ends with the ENDP directive.

Example

```

0000          SUMS  PROC  NEAR
0000 03 C3      ADD  AX, BX
0002 03 C1      ADD  AX, CX
0004 03 C2      ADD  AX, DX
0006 C3         RET
0007          SUMS  ENDP

0007          SUMS1 PROC  FAR
0007 03 C3      ADD  AX, BX
0009 03 C1      ADD  AX, CX
000B 03 C2      ADD  AX, DX
000D CB        RET
000E          SUMS1 ENDP

000E          SUMS3 PROC  NEAR USE BX CX DX
0011 03 C3      ADD  AX, BX
0013 03 C1      ADD  AX, CX
0015 03 C2      ADD  AX, DX
                RET
001B          SUMS  ENDP

```

Example

0000		SUMS	PROC	NEAR	
0000 03 C3			ADD	AX, BX	Removes 16 bit from stack
0002 03 C1			ADD	AX, CX	
0004 03 C2			ADD	AX, DX	
0006 C3			RET		
0007		SUMS	ENDP		
0007		SUMS1	PROC	FAR	
0007 03 C3			ADD	AX, BX	Removes 32 bit and puts in CS & IP
0009 03 C1			ADD	AX, CX	
000B 03 C2			ADD	AX, DX	
000D CB			RET		
000E		SUMS1	ENDP		
000E		SUMS3	PROC	NEAR USE BX CX DX	
0011 03 C3			ADD	AX, BX	
0013 03 C1			ADD	AX, CX	
0015 03 C2			ADD	AX, DX	
			RET		
001B		SUMS	ENDP		

Example

```
SUM PROC NEAR USES BX CX DX ; Save BX, CX, DX automatically  
    ; Procedure body  
    MOV BX, 10  
    MOV CX, 20  
    ADD DX, CX  
    ; Registers BX, CX, DX will be restored automatically  
    RET  
SUM ENDP
```

Example

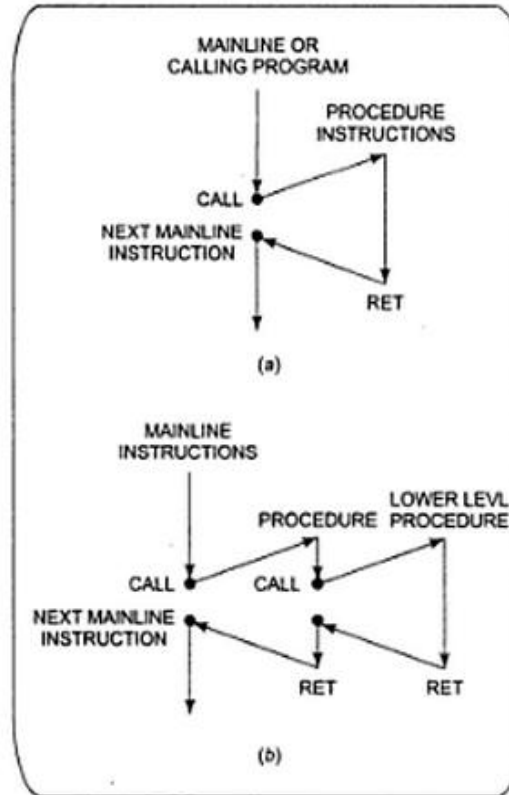
```
SUM PROC NEAR USES BX CX DX ;  
    ; Procedure body  
    MOV BX, 10  
    MOV CX, 20  
    ADD DX, CX  
    ; Registers BX, CX, DX will  
    RET  
SUM ENDP
```

```
SUM PROC NEAR  
    PUSH BX    ; Manually save registers  
    PUSH CX  
    PUSH DX  
  
    ; Procedure body  
    MOV BX, 10  
    MOV CX, 20  
    ADD DX, CX  
  
    POP DX    ; Restore registers manually  
    POP CX  
    POP BX  
    RET  
SUM ENDP
```

Call

- The CALL instruction differs from the jump instruction because a CALL saves a return address on the stack.
- The return address returns control to the instruction that immediately follows the CALL in a program when a RET instruction executes.

Call & Return



Types of CALL

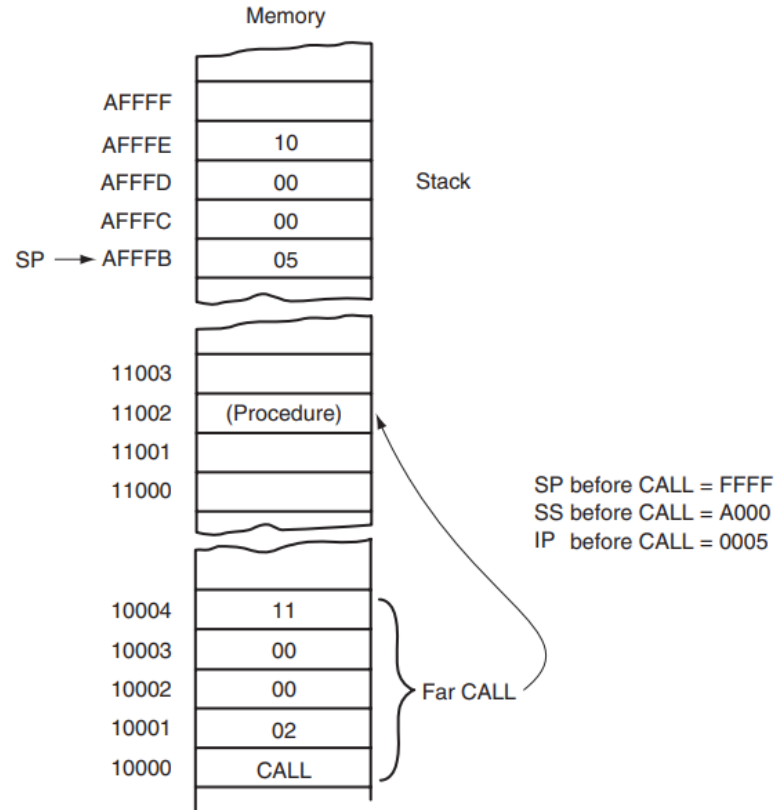
Near CALL

- Near CALL instruction is 3 bytes long; the first byte contains the opcode, and the second and third bytes contain the displacement, or distance of $\pm 32K$

Far CALL

- 5-byte instruction that contains an opcode followed by the next value for the IP and CS registers. Bytes 2 and 3 contain the new contents of the IP, and bytes 4 and 5 contain the new contents for CS.

Far CALL



Types of Calls

CALLs with Register Operands

- contains a register operand.

E.g. CALL BX instruction, which pushes the contents of IP onto the stack. It then jumps to the offset address, located in register BX, in the current code segment

CALLs with Indirect Memory Addresses

- Useful whenever different subroutines need to be chosen in a program.

E.g. CALL TABLE [4*EBX]

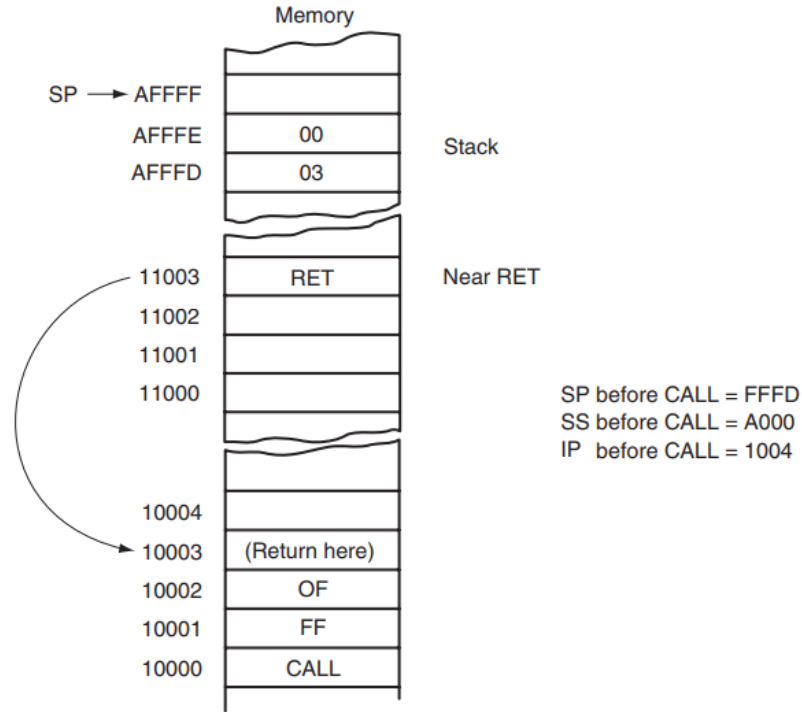
RET

- The return instruction (RET) removes a 16-bit number (near return) from the stack and places it into IP, or removes a 32-bit number (far return) and places it into IP and CS.
- The near and far return instructions are both defined in the procedure's PROC directive, which automatically selects the proper return instruction.

RET



BITS Pilani
K K Birla Goa Campus



Example

```
MOV    AX,30
MOV    BX,40
PUSH   AX                ;stack parameter 1
PUSH   BX                ;stack parameter 2
CALL   ADDM              ;add stack parameters
```

```
ADDM   PROC    NEAR
        PUSH   BP                ;save BP
        MOV    BP,SP             ;address stack with BP
        MOV    AX,[BP+4]         ;get parameter 1
        ADD    AX,[BP+6]         ;add parameter 2
        POP    BP                ;restore BP
        RET    4                 ;return, dump parameters
ADDM   ENDP
```



BITS Pilani
K K Birla Goa Campus

THANK YOU

